

Facade (Fasada)

czyli jak przestać się męczyć z milionem klas

Wzorce projektowe – laboratorium

10 października 2025

Wyobraź sobie...

Chcesz zamówić pizzę przez telefon.

Chcesz zamówić pizzę przez telefon.

- Dzwonisz do magazynu: "Czy macie ser i salami?"

Chcesz zamówić pizzę przez telefon.

- Dzwonisz do magazynu: "Czy macie ser i salami?"
- Dzwonisz do księgowości: "Proszę zaksięgować 35 zł"

Chcesz zamówić pizzę przez telefon.

- Dzwonisz do magazynu: "Czy macie ser i salami?"
- Dzwonisz do księgowości: "Proszę zaksięgować 35 zł"
- Dzwonisz do kuriera: "O 18:00 na ul. Długą 5"

Chcesz zamówić pizzę przez telefon.

- Dzwonisz do magazynu: "Czy macie ser i salami?"
- Dzwonisz do księgowości: "Proszę zaksięgować 35 zł"
- Dzwonisz do kuriera: "O 18:00 na ul. Długą 5"
- Dzwonisz do działu marketingu: "Dolicz mi punkty lojalnościowe"

Chcesz zamówić pizzę przez telefon.

- Dzwonisz do magazynu: "Czy macie ser i salami?"
- Dzwonisz do księgowości: "Proszę zaksięgować 35 zł"
- Dzwonisz do kuriera: "O 18:00 na ul. Długą 5"
- Dzwonisz do działu marketingu: "Dolicz mi punkty lojalnościowe"
- Dzwonisz do SMS-ów: "Wyślij potwierdzenie"

Chcesz zamówić pizzę przez telefon.

- Dzwonisz do magazynu: "Czy macie ser i salami?"
- Dzwonisz do księgowości: "Proszę zaksięgować 35 zł"
- Dzwonisz do kuriera: "O 18:00 na ul. Długą 5"
- Dzwonisz do działu marketingu: "Dolicz mi punkty lojalnościowe"
- Dzwonisz do SMS-ów: "Wyślij potwierdzenie"

Absurd? Dokładnie tak wygląda kod bez Facade.

Bez Facade – kod kliencki

```
# Klient chce zamowic pizze...
inventory = InventoryManager()
if not inventory.check_availability("Margherita"):
    print("Brak pizzy!")
    return

payment = PaymentProcessor()
if not payment.process_card("1234-5678", 35.0):
    print("Platnosc odrzucona!")
    return

delivery = DeliveryScheduler()
delivery.schedule("ul. Długa 5", "18:00")

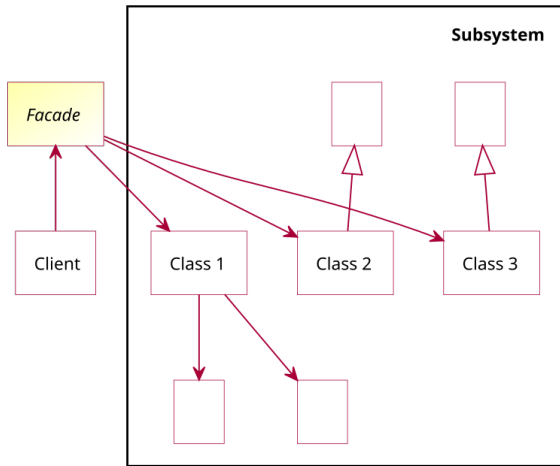
loyalty = LoyaltyPointsCalculator()
loyalty.add_points("user123", 35.0)
notifications = NotificationService()
notifications.send_sms("user123", "Zamowienie potwierdzone!")
print("Uff... w koncu!")
```

Problem

- Klient musi znać **wszystkie** klasy podsystemu
- Musi znać **kolejność** wywołań
- Musi **obsługiwać błędy** na każdym kroku
- Kod jest **nieczytelny** i **powtarzalny**
- Zmiana w podsystemie = zmiany w **wielu miejscach**

Rozwiązanie: Facade

Wzorzec Facade (WikiCommons)



Sample class diagram

Z Facade – kod kliencki

```
# Klient chce zamowic pizze...
facade = PizzaOrderFacade()

result = facade.place_order(
    pizza_type="Margherita",
    address="ul. Długa 5",
    delivery_time="18:00",
    card_number="1234-5678",
    user_id="user123"
)

if result.success:
    print("Zamowienie zlozone!")
else:
    print(f"Bład: {result.error}")
```

Wow. Czytelne, proste, łatwe w utrzymaniu.

Kiedy używać Facade?

- Masz **skomplikowany podsystem** z wieloma klasami
- Klient potrzebuje tylko **najprostszych operacji**
- Chcesz **odizolować** klienta od szczegółów implementacji
- Chcesz **uproszczyć API** dla zewnętrznych użytkowników

Trade-off: Facade może stać się "god object" jeśli wrzucisz do niej za dużo.

Dzisiaj na zajęciach

- 1 Dostaniecie kod systemu zamówień pizzy (bez Facade):
`https://github.com/refactor-or-die/lab01-pizza-driven-development`
- 2 W parach zrefaktoryzujecie go, dodając PizzaOrderFacade
- 3 Zaprezentujecie swoje rozwiązanie

Pytania?

Jeśli nie – to `git clone` i do roboty!